

MockGen: Realistic Offline Mock Data for Enterprise Applications

Christos Koutsiaris
SAP P&E – Cloud ERP – UX Foundation
Development Expert
christos.koutsiaris@sap.com

Abstract—Developers of enterprise applications need screens full of realistic data long before any backend exists. Hand-written mock data is expensive and goes stale; random generators produce values that look broken on screen. MockGen generates complete, relationally consistent mock datasets directly from an application’s service definition, offline and deterministically. Every column is resolved by a cascade: declared catalogs, then a recognised semantic role, then a small local language model, then a type-correct fallback that cannot decline. Recognition is a learned classifier trained on 23,741 fields labelled by a panel of three large language models that see only structural metadata. On 131 real service definitions the pipeline generates 11,529 fields with 0 failures and binds 44.79% of them to a semantic provider; with recognition disabled the same corpus binds 22.93% and 12 services fail outright. On 41 services held out from training, the classifier’s decisions are correct 96.2% of the time across 44 roles. We also report a verifier that did not work and what we shipped instead.

Index Terms—mock data, test data generation, service metadata, OData, offline inference, semantic role classification, relational data generation

I. INTRODUCTION

Afrontend developer starts from a service definition — a contract naming entities, columns, types, widths and relationships — and needs data on screen before any backend exists. Hand-written mock data drifts out of sync with the schema. Random generators fill a city column with random words, so every screen looks broken. Recording real responses needs access, permissions and cleanup.

Enterprise schemas amplify this. A service declares dozens of entities and hundreds of columns with exact widths; some columns are protocol artifacts rather than business data; annotations encode meaning a generator should honour. Errors are visible: a too-long value truncates, a code that does not match its text reads as a bug, an empty child table makes the application look defective.

MockGen generates a complete dataset from the service definition alone: offline, deterministic, in seconds. Five rules govern it: any application works cold; no screen is empty; data looks real at a glance; no internal failure blanks the application; and the same input always produces the same output. The architecture is a *tier cascade* (Fig. 1): the first tier that can produce a valid value wins, and the last tier cannot decline.

Contributions. (i) The four-tier cascade and the schema representation behind it (Sections II–III); (ii) a learned role

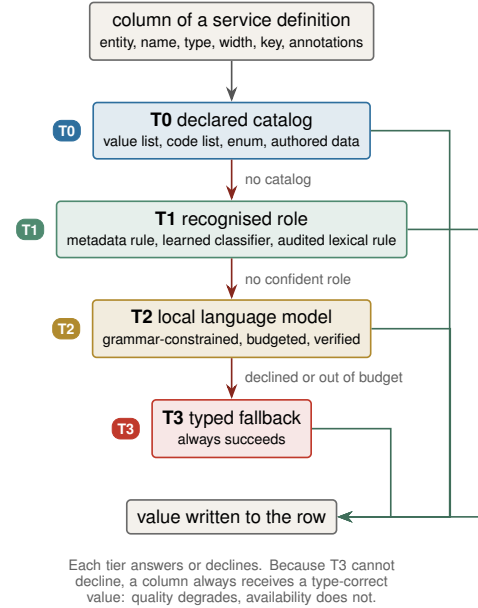


Fig. 1. The cascade for one column. Declared catalogs beat recognition, recognition beats the language model, and the typed fallback cannot refuse. Takeaway: the never-empty promise is structural.

recogniser trained on machine-adjudicated labels, with a routing policy that decides per role whether the model may speak (Sections IV–V); (iii) a measured characterisation of the shipped system, including a negative result (Sections VI–VII).

II. SCHEMA IR AND ADAPTERS

Everything downstream reads one internal shape: a schema graph holding entities, properties with type, width, precision, key and nullability flags, declared labels, data elements and descriptions, relationships with their column mappings, and annotation-derived links such as value lists and text associations. Adapters translate each input format into it, so the tiers never learn protocol dialects. Two ship today: the metadata format (EDMX) of Open Data Protocol (OData) services, and Core Data Services (CDS) documents.

III. THE FOUR TIERS

T0, declared catalogs. If the schema declares allowed values, or the developer supplied authored rows, the value comes

from there. A value-list annotation points at a code entity; the generator produces that code table first and draws from it, so a code column and its text column agree by construction.

T1, recognised roles. Otherwise the system recognises what the column means and draws from a curated bank. A role declares the types it accepts, whether it may sit on a key, its provider, its validator, its coherence group and its routing confidence. Roles group into families — status, identity, location, finance, temporal, narrative, contact, organisation, measure — and families decide both support floors and error severity.

Three sources feed one arbitrator. Metadata rules fire when an annotation or data element settles the question, and win outright. The learned classifier handles the rest. Audited name-based rules remain where a name is conclusive; each was checked against adjudicated labels, and the eight that contradicted a label with a different positive role were switched off. When classifier and lexical rule disagree, the classifier wins only with a singleton prediction set and a stricter confidence; otherwise both are discarded. Compatibility is enforced last, so a role forbidden on keys never lands on one. Draft-administration and action-control columns abstain unconditionally.

T2, a budgeted local language model. For visible text with no recognised role, a small quantized model proposes values under a grammar and a wall-clock budget, on two virtual central processing units (vCPUs). When the budget expires the tier yields and the cascade continues.

T3, the guaranteed floor. A typed generator emits values respecting type, width, precision and nullability, unique where uniqueness is required. It cannot decline, which is what turns “no empty screens” into a property.

Relational passes. Keys are generated first and kept unique within their entity. Child rows draw foreign keys from parents that were actually generated, so references resolve. Coherence groups — city with country and postal code, amount with currency — are generated together, and a second pass recomputes derived display values. Datasets are cached under a key including the schema, seed, row counts and every model fingerprint, so a stale dataset is never served after a model swap.

IV. LEARNING TO RECOGNISE A COLUMN

A. Labels without human annotators

We use a panel of three large language models from two vendors: Claude Opus 4.8, Claude Sonnet 4.5 and GPT-5.6-terra. Each sees one field’s structural metadata and a frozen guideline, and returns a registered role or `unknown`. Judges are independent; a field is labelled when at least two of three agree, and disagreements are excluded rather than tie-broken. six rounds produced 23,741 labels, with Fleiss’ kappa from 0.87 down to 0.72 on a round aimed at the hardest cases and back to 0.81 after the guideline was clarified. Every label records machine panel consensus with `humanVerified` false; we claim consistency and reproducibility, not parity with expert labels.

Services group into families, and a family belongs to exactly one partition: 12,375 training rows from 119 groups, 4,991 calibration rows from 39, and a sealed set of 4,317 fields across 41 services. Contexts with conflicting labels, and contexts appearing in two partitions, are dropped rather than relabelled.

B. Two findings that mattered

The classifier embeds a text rendering of the field context with a frozen sentence encoder (Fig. 2). The pilot rendered a delimited record, `v3|BookingStatus_code|string|...`, which is a fine machine format and a poor input for an encoder trained on prose. The shipped rendering is a sentence: `Booking Status code (string) in Travel Booking; data element BOOK STATUS; a code field.`

Identifiers become words, structural facts are stated in words, and verbose material that fills the 64-word-piece budget without carrying role evidence is dropped. With the same encoder and labels, calibration accuracy rose from 0.63 to 0.84. The final clause names the identifier’s head noun, because mean pooling cannot weigh the last word yet that word decides meaning; it raised status routing precision from 45/56 to 91/100.

The second finding is capacity. One hidden layer of 256 rectified-linear units on the same embeddings raised calibration accuracy from 0.62 to 0.84 and the routed share of role-bearing calibration fields from 29% to 74%. The shipped head is a multilayer perceptron (MLP) of $384 \times 256 \times 63$, adding 3.4 MiB to the package.

V. WHEN THE CLASSIFIER MAY SPEAK

Three gates. **Support:** at least 5 correct calibration examples for the role and 10 for its family. **Measured precision:** each role’s threshold is raised — never lowered — to the lowest confidence reaching 0.95 precision on calibration with enough decisions to measure; a role that cannot reach it does not route. **Conformal set size:** a decision whose prediction set holds more than one class abstains, at a quantile targeting 0.900 coverage.

Roles failing these gates stay as *auxiliary* classes: trained, never selected. When we instead removed weak classes from training, status-text columns had nowhere to go and were misrouted as status codes. Of 63 classes, 44 route and 18 are auxiliary, so coverage grows with evidence rather than with code changes.

VI. RESULTS

A. Corpus

We run the whole pipeline over every checksum-verified service in the source registry whose format the package parses directly: 131 of 473 registered, covering 1,345 entities and 11,529 fields at two rows per entity. All 131 generate, with 0 failures. 44.79% of fields bind to a semantic provider: classifier 29.01%, metadata rules 5.96%, lexical rules 11.85%.

Disabling recognition isolates the learned tier (Fig. 3): binding falls to 22.93% and, unexpectedly, 12 services fail

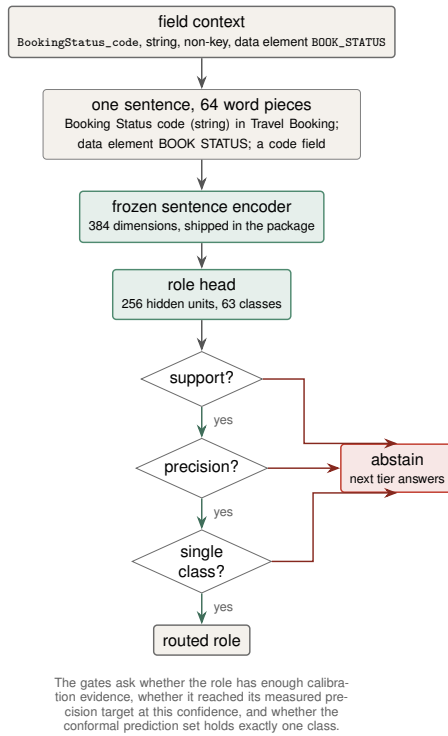


Fig. 2. One column becoming a routed role: sentence rendering, frozen encoder, small head, three gates that can each fall back to abstention. Takeaway: routing is a decision about trust, not the top-scoring class.

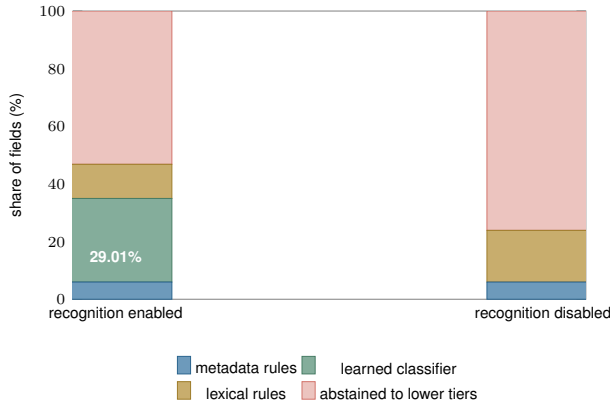


Fig. 3. Field resolution across the 131-service corpus with and without the learned tier. Takeaway: the classifier roughly doubles semantic binding, 22.93% to 44.79%.

outright, because without a recognised role some code columns cannot satisfy their own semantic validation. Recognition is a robustness feature as well as a realism one.

B. Recognition quality on unseen services

Corpus shares say how often the classifier speaks, not whether it is right. The sealed set is family-disjoint from training and calibration and is scored through the real runtime path. On 41 services and 21 domains never seen, the classifier

TABLE I
SEALED EVALUATION ON 41 UNSEEN SERVICES (4,317 FIELDS). PRECISION COUNTS THE CLASSIFIER’S OWN DECISIONS; DETERMINISTIC RULES ARE LISTED SEPARATELY. TAKEAWAY: WHEN THE MODEL SPEAKS IT IS RIGHT 96.2% OF THE TIME.

Measure	Value
Classifier routing decisions	1,206
correct	1,160 (96.2%)
critical false positives	27 (2.2%)
Distinct roles routed	44
Supported-field recall	63.0%
Status fields detected	27/44
classifier’s top label correct	33/44
Decisions from metadata rules	389
Decisions from lexical rules	512

makes 1,206 decisions of which 1,160 are correct (96.2%) across 44 roles, with a critical false-positive rate of 2.2%.

Two numbers belong together: status recall is 27/44 while the top label is correct on 33/44. The first gap is the routing policy declining what it cannot justify; the second is the model being wrong, and only that closes with more labels. Residual errors concentrate on one boundary — a generic caption against a specific named thing — the same boundary where the judges themselves needed a guideline revision.

C. Speed and footprint

Generation time is dominated by fixed costs, not dataset size (Fig. 4): 17 rows cost 270 ms and 1,700 rows 453 ms on a 17-entity service, from 15.9 down to 0.27 ms per row. Median service time is 107 ms without recognition and 1,442 ms with it; both isolate recognition by giving the language model a budget it cannot use, and letting it run at its packaged budget raises the median to 3,808 ms for the same resolved fields. The reference application with every tier enabled generates in 2,845 ms.

The published archive is 128 MB against a 200 MB registry limit: encoder 21.9 MiB, role head 3.4 MiB, vocabulary 0.2 MiB, relevance head 0.1 MiB, generation model 157.3 MiB. The editor extension (11.4 MB) carries no models: it pins the package by version, application programming interface (API) version and hashes, and resolves that pin once into a shared verified cache. The build passes 466 package tests, 80 training-tool tests and 50 editor tests with 0 lint errors.

VII. A COMPONENT THAT DID NOT WORK

Before a model-proposed display value is written, a verifier should judge whether it suits the field; the intended gate was 80% of valid candidates accepted at no more than 1% of wrong ones. We built 3,346 panel-judged pairs over 19 approved value sources with 19 held out, and tried a frozen-encoder logistic head, a fine-tuned bi-encoder and a cross-encoder. None separated the classes: at the 1% budget the verifier accepts 1.3% of valid candidates, and loosening it enough to be useful admits about 14% of clearly-wrong values. Expanding the corpus made the task harder, and the panel judged a sixth of

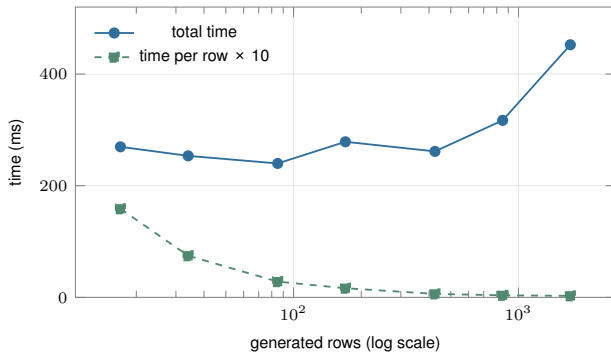


Fig. 4. Deterministic-pipeline scaling on a 17-entity service. Takeaway: everything except the language model is effectively free at realistic dataset sizes.

our supposed wrong values acceptable — the task as posed was not well defined.

We ship the strict point (5/399 valid, 6/704 wrong). Because that rejects nearly everything and the pipeline treated “unverified” as fatal, applications with synthetic display domains produced no data. Generation now keeps the deterministic values, marks the resource unverified and emits a diagnostic; the guardrail for a *missing* verifier is unchanged.

VIII. DISCUSSION AND LIMITATIONS

The pilot recogniser routed nothing useful for three reasons, none of them the model: the input was written for a machine rather than the encoder, the head lacked capacity for the class count, and a registry default threshold overrode the head’s own calibration so decisions between 0.6 and 0.9 never routed. Each was invisible in aggregate accuracy and obvious once every sealed decision recorded its confidence, threshold, prediction-set size and abstention reason.

Machine labelling bought scale — 23,741 labels for the cost of compute — but is also the ceiling: residual errors sit where inter-judge agreement was lowest.

Limitations: labels are machine consensus, not human-verified; corpus numbers count fields bound to a provider, not fields a human would judge realistic; the corpus is 131 services of 473 registered, and the sealed set, while family-disjoint, comes from the same registry; supported recall is 63.0%; the verifier is nearly inactive; and gate thresholds were revised once, with the owner’s approval, when the sealed set grew from 45 fields to 4,317, with both values recorded in the release baseline.

IX. CONCLUSION

MockGen turns a service definition into a complete, coherent, deterministic mock dataset with no backend, no network and no human labelling effort. On 131 real services it generates 11,529 fields with 0 failures and binds 44.79% to a semantic provider, against 22.93% and 12 failures without recognition; on 41 unseen services its learned decisions are correct 96.2% of the time across 44 roles.

GLOSSARY

- API Application programming interface.
- CDS Core Data Services, SAP’s schema definition language.
- Conformance-driven method producing a set of plausible classes at a target coverage level.
- EDMX The XML document format for OData service metadata.
- Fleiss’ kappa Agreement measure for more than two raters, corrected for chance.
- MLP Multilayer perceptron, a network with a hidden layer.
- OData Open Data Protocol, used by SAP Fiori applications.
- ONNX Open Neural Network Exchange, the packaged model format.
- vCPU Virtual central processing unit.
- VSIX The package format for a code editor extension.